:: CHAPTER # 3 ::

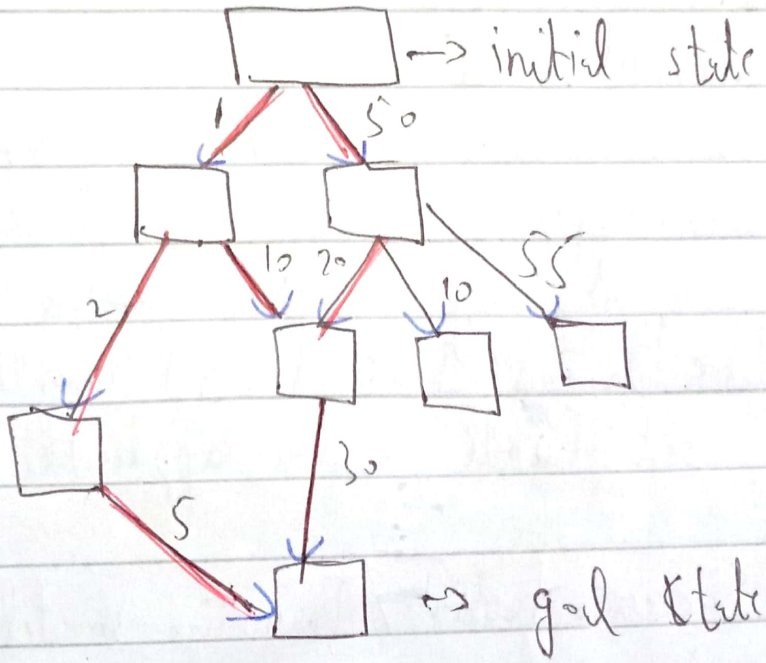
: State-Space-Graph :

State-space:

A set of all states reachable by the initial state by following any sequence of actions is called, the state-space.

State-space-graph:

A state-space graph is the mathematical representation of a search problem.



-> The nodes represent the states
-> Links b/w nodes are actions with a cost

Scanned with CamScanner

-> The red paths are sequence of actions that leads towards goal state.
-> As we can see there are many action sequences and we choose the one which has minimum cost.

:: Well-defined - Problem ::

A well defined problem has the following ~~four~~ five components :-

-> Initial state
-> Possible actions :-

A description of all possible actions available for agent e.g given ~~at~~ a state s, ACTION(s) will return set of all actions applicable on state s.

-> Successor function / Transition model :-

Successor gives Function or transition model ~~it~~ the state(s) that is achieved by doing an action.

Scanned with CamScanner

-> Goal Test :

A goal test determines whether the given state is goal state or not

-> Path cost Fuction :

A path cost fuction assigns a numeric cost to each path.

Solution of problem :

A solution to a problem is a sequence of actions that leads towards the goal state.

: Vaccum - World - Problem :

See the ~~state~~ space graph on P-g 7.

initial state :

Any state can be initial.

Actions / operator :

left, right, suck

Successor fuction / transition model :

A state depends on actions ~~i.e. ff~~

Scanned with CamScanner

Goal test:

This check whether all squares are clean

Path cost:

going to any state cause 1 cost so total cost is the number of actions in the path.

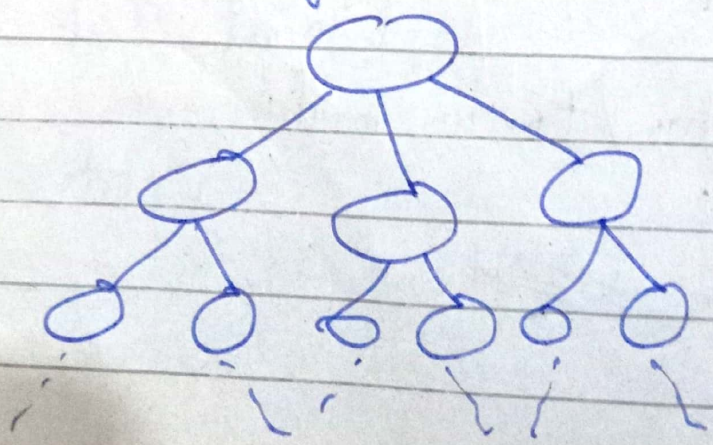
:: TREE - SEARCH ::

→ A search tree is used to find solution of search problem. Its properties are as follows.

→ Root represents the initial state

→ Branches / links represents the actions

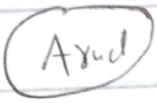
→ Nodes represent the state we reach by doing an action.



Scanned with CamScanner

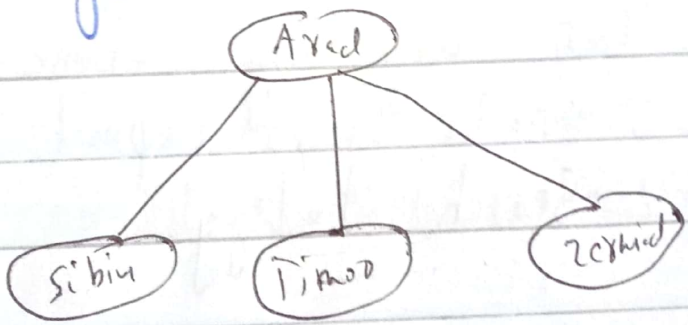
How tree search works :-

First we are on the initial state, we take example of Romania from book



-> We check whether it is goal state or not if it is not a goal state

-> We expand the current state by applying each legal action on the current state and hence we get a new set of states.



-> Now we have three leaf nodes and now we consider any of them for further expansion, in case the first doesn't lead to solution we select

Scanned with CamScanner

other leafs for expansion.

-> The set of all leaf nodes available for expansion is called the Frontier

-> Now this process of expanding the states in frontier continues until we find the solution or there is no further state to expand.

-> Almost all search tree algorithms follow the above steps, the main difference is that how they choose the next node to expand is called the Search strategy.

Scanned with CamScanner

Pseudo Code:

solution/fail

FUNCTION TREE-SEARCH (Problem) Return

Initialize (frontier) with initial state

Loop do

1 if frontier empty then return Failure

2 choose a node on frontier and remove it.

3 if isGoal(node) return corresponding solution

4 Expand node and add resulting nodes in frontier

Problem in Tree Search:

The main problem that make the search tree ∞ and make the solvable problem unsolvable are the loopy path so while constructing the search tree we should ignore loops.

Scanned with CamScanner

:: Infrastructure For a Search algorithm ::

-> Search algorithms requires some sort of structure to keep track of tree that is being constructed.

-> For each node n we require a structure that contains the following four components.

•) n.State:
the state in the state space to which the node corresponds

•) n.Action :
the action that was applied on the parent to achieve the node.

•) n.Parent :
a pointer to the node in tree that generate this node.

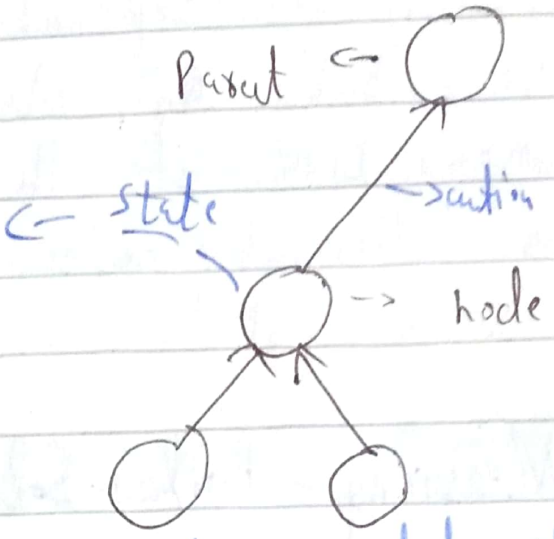
Scanned with CamScanner

(1, 4)

- n.path_cost :

The cost from initial state to this node.

5	6	1
8	4	2
9		3



The child node can be created if we have info of parent and the action.

Function CHILD-NODE (parent, action) returns node
initialize a node n
n.state = Successor-fn (parent.state, act)
n.parent = parent
n.Action = action
n.path cost = parent.path cost + COST (parent.state, act)

return n

Scanned with CamScanner

:: Difference b/w node & state

The node ~~are the~~ is a structure that is used to represent the search tree but a state is the configuration of the enviroment

-> Two different nodes can contain same state.

:: Measuring - Problem-Solving - Performance ::

The performance of a search algorithm is measured in four ways.

-> Completeness : Is algorithm guranteed to find solution if it exists

-> Optimality : Does it find the optimal one

-> Time Complexity

-> Space Complexity

Scanned with CamScanner

How to measure complexities :-

In theoritical computer science the complexity is measured by the size of the state space graph ($|V| + |E|$). This strategy is applicable when we already have a complete graph like in Romania problem and we input this graph to search program.

-> But in AI, the graph is represented by the initial state, actions and transition model and can be infinite and we do not know exact size of graph in advance so under such situation ~~we~~ Complexity is expressed in terms of three quantities.

-> branching factor (b) :-
Maximum number of successors of a node.

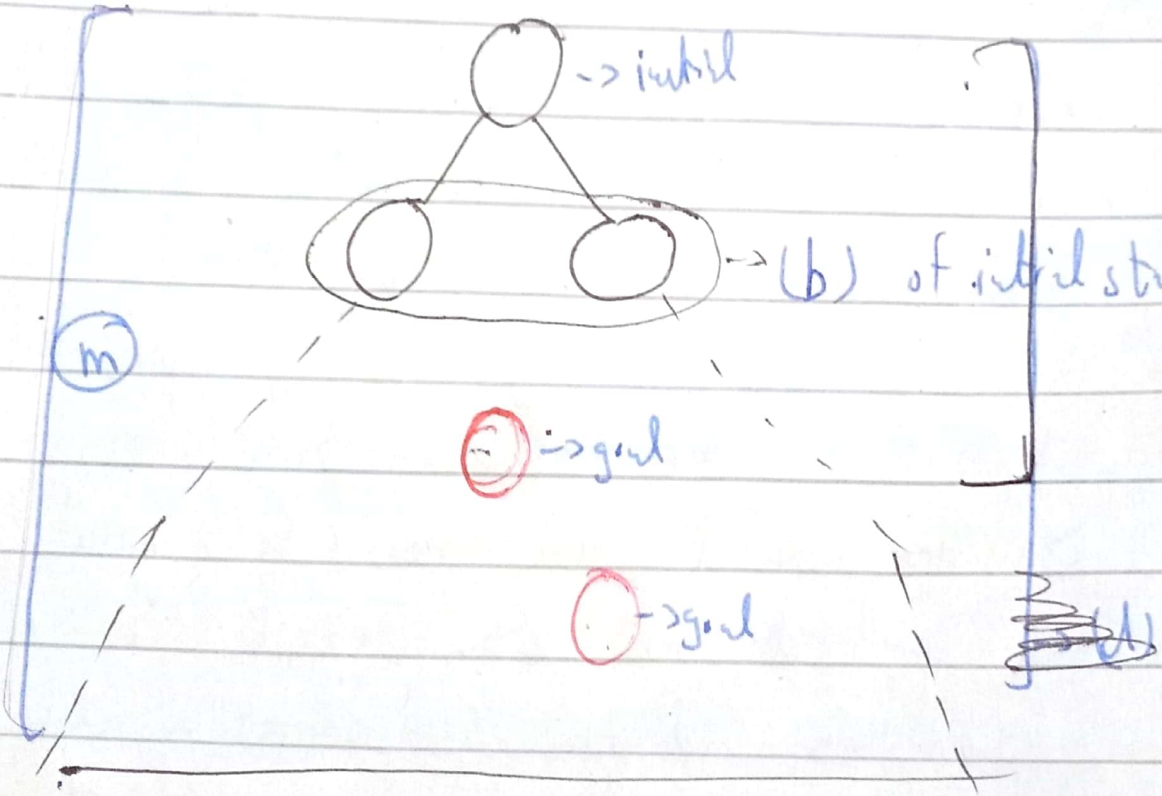
-> Depth (d) :-
Depth of the shallowest goal.

-> m :-
Maximum length of any path.

Scanned with CamScanner

-> Time is measure in terms of nodes generated during search

-> Space is measured in terms of nods stored in memory



Scanned with CamScanner

:: Un-informed - SEARCH ::

-> In uniformed search or blind search we have no additional info about states and we expand all states until we reach a goal state.
that is we have no mechanism that tell us that one non-goal state (expanding) is more beneficial than other.

-> BFS
-> Uniform - Cost - search
-> DFS
-> Depth limited search
-> Iterative deepening search

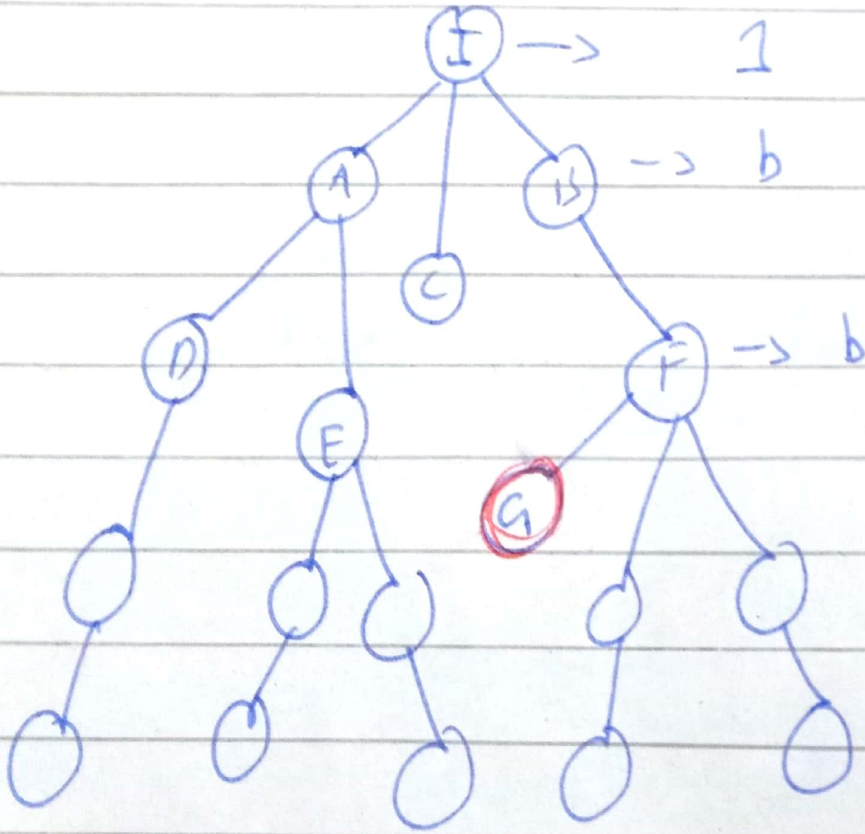
Scanned with CamScanner

Breadth-First-Search :-

-> In BFS we explore all nodes of one level before going to next level.

-> We use a queue (FIFO) in our Frontier

-> let



Scanned with CamScanner

we have following BFS tree search algo

```
Function BFS (problem) return solution + failure
    initialize a queue
    Push initial node in queue
    loop do, if (Q.empty) return failure
        node = Q.pop()
        if (Goal test (node)) return solution
    for each a in ACTIONS (node.state) do
        child ← CHILD NODE (problem, node, a)
        Q.push (child)
```

Completeness :-

of course - BFS satisfies completeness.

Optimality

As BFS returns the shallowest node that is a solution so it is optimal provided the cost reaching to a node is same for all.

Scanned with CamScanner

Time complexity :

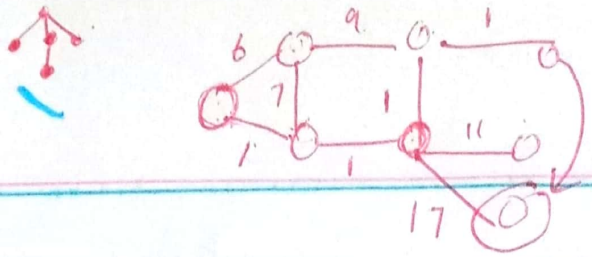
Let b is the branching factor of a node, and d is depth of shallowest goal so we the nodes generated (Time complexity)

$= 1 + b + b^2 + b^3 \ldots\ldots b^d = \boxed{O(b^d)}$

Space Complexity :

$\boxed{O(b^d)}$

Scanned with CamScanner



:: Uniform - Cost - Search ::

→ We see that BFS is optimal when all step costs are equal.

→ If all step costs are not equal then we'll use UCS.

→ In UCS instead of expanding the shallowest node. We expand the node which has the lowest path cost.

→ It is uses min priority queue.

Pseudo Code ::

```
Function UCS (Problem) return solution / failure
    node ← problem.initial state
    frontier ← priority queue
    explored ← a set of explored states
    Loop do
        if (frontier.empty()) return failure
        node ← frontier.pop
        if (node.state == goal state(prob))
            return solution
        explored.add(node.state)
```

first diff

problem.

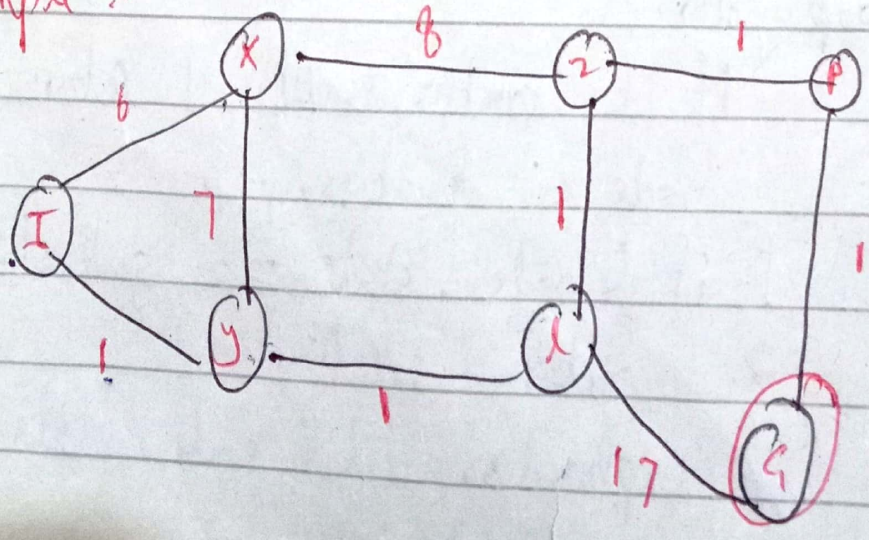
for each a in Actions(node.state) do
child ← Child-NODE(problem, node, a)
if child.state not in explored or frontier
Frontier.push(child)

Second diff ← else if child.state is in Front with high cost
replace that Frontier node with child.

∴ Key diff b/w UCS & BFS in addition to priority Queue :-

→ The first major difference is that the GOAL-TEST is applied when a node is selected for expansion rather when it is first generated because. when a node is generated it may be on a suboptimal path.

Example :-



Scanned with CamScanner

Consider the graph now when we apply UCS on the graph we start from I and we follow a path with costs as

$$I \rightarrow J \rightarrow L$$

now when children of L are generated the goal node also generates and if we apply goal test here we will get the following solution

$$I \xrightarrow{1} J \xrightarrow{1} L \xrightarrow{17} G = 19$$

Which is not an optimal solution, because the optimal is

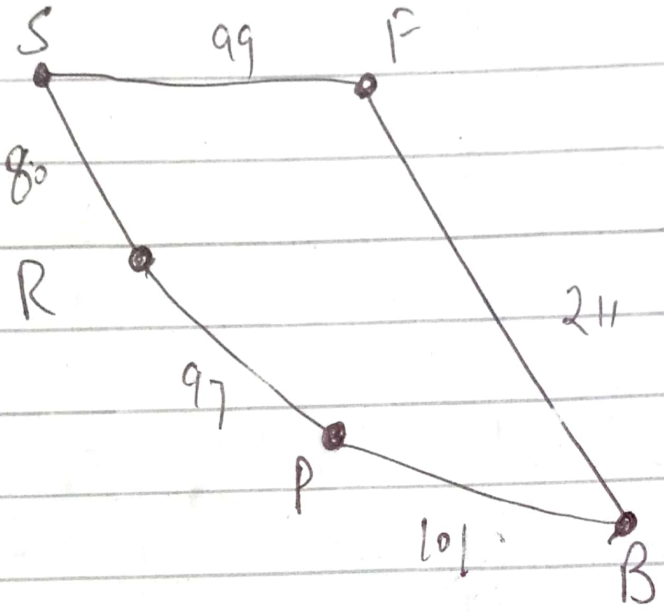
$$I - J - L - R - P - G = 5$$

This solution will achieve only if we select a node for goal test when it is selected for expansion rather when it is generated.

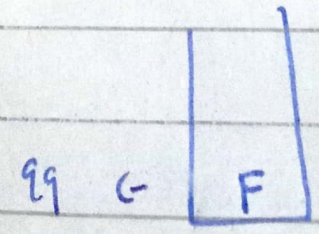
Scanned with CamScanner

→ Second difference is that when a lower path cost is calculated for a state that is already in the frontier then it is replace with current node.

Example:



In this example our start state is S and goal state is B. now first we goto S → R with cost 80



Scanned with CamScanner

on R we generate its child and put them in queue

80+97 | P
99 | F

~~we go from P to B with cost~~
~~80+97+10~~

Now we expand F and reach to goal state with cost 99+211 = 310
but we not apply goal test here
now we expand P and again reach to goal state B in cost ~~80~~ 278 which is less than the cost that is already in frontier so we replace them.

Scanned with CamScanner

Problem with UCS :-

It will get stuck into an infinite loop if ∃ a path with ∞ sequence of zero-cost actions.

:: Analysis ::

Optimality :

Yes.

Completness

yes if cost of every step is > than a small +ve number ε

Time Complexity :-

$b^{1+\lfloor C/\varepsilon \rfloor}$ > BFS

Scanned with CamScanner

:: Depth-First-Search ::

-> In DFS we expand search tree to it its maximum depth and then back track.

-> DFS uses LIFO structure (stack)

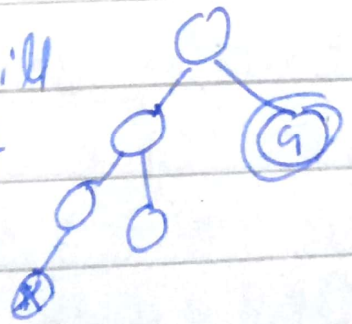
-> DFS may stuck in ∞ loop so to avoid it we keep track of visited nodes.

-> It pseudocode is like BFS but frontier is LIFO.

-> :: Analysis ::

Optimality :

No, as it will first go to x and after that it go to G.



Completness :

No.

Time Complex $O(b^m)$

Space $O(bm)$

Scanned with CamScanner

Backtracking Search:-

A variant of DFS is backtracking search in which we only keep one successor in memory & space complexity become $O(m)$.

Depth-limited-Search

→ DLS ~~resolve~~ kindof resolve the ∞ depth problem.

→ It sets a depth l for exploring the tree and do not go further when depth limit l is reached.

→ This search is incomplete if $l < d$

→ This search is not optimal if $l > d$

→ It time complexity is $O(b^l)$

→ It space complexity is $O(bl)$

DFS can be treated as DLS with $l = \infty$

Scanned with CamScanner

-> In DLS in addition to standard failure, there is another cut off failure meaning that limit is reached.

Pseudo Code

```
Function DLS (problem, limit) return solution, f, cutoff
    return RDLS (node, problem, limit)

RDLS (node, problm, limit)
    if problem.goal test(node.state)
        return solution
    if limit = 0 return cutoff
        cutof occur = false
        For each actin in problem.Actions do
            child node
            result = RDLS(nodchild, prob, l-1)
            if result, cutoff cutof occ=t
            if result ≠ Failure return result
    if cutof occ = T return cutof
        else failure.
```

Scanned with CamScanner

Informed Search Strategies :-

-> In informed search strategies, we have some additional Knowledge of the problem (heuristic) beyond the problem statement.

-> The Knowledge is determined by the heuristic fuction whose definition is

Heuristic function :-

An heuristic $h(n)$ = estimated cost of the cheapest path of state at node n to goal state.

Scanned with CamScanner

: Greedy - Best - first Search :-

Greedy best first search is just like UCS but it selects the node for expansion that has the smallest h(n) value.

→ For explanation we have contarted a heuristic table which contains the Euclidean distance of each city to Bucharest (goal state).

→ Starting from Arad we go to sibiu because h(in(sibiu)) = 253 which is smallest among Zerind, Timisoara and sibiu. In this way we have a solution that is higlighet in green.

Arad → sibiu → Fagaras → Bucharest

Scanned with CamScanner

:Analysis of algorithm:

Optimality:

The algorithm uses the greedy which is not always optimal because as we see in map of Romania problem the total cost of solution is 450, but optimal cost is 418 which is highlighted in blue Hence it is not optimal in general

Completeness:

The tree version of the algorithm is not complete in finite state spaces.

e.g:

If we start from Iasi according to heuristic table we go to Neamt and from neamt we have a dead end or we go back to Iasi and from Iasi we go again to Neamt and we got stuck in an ∞ loop.

Scanned with CamScanner

Time complexity & space complexity :-

It time complexity is (b^m)

Scanned with CamScanner

A* Search

-> We can say that A* search is the combination of both UCS and greedy best first search.

-> It selects a node n on the frontier that has the least f(n) value

$$f(n) = g(n) + h(n)$$

g(n) = the cost to reach n from initial node

h(n) = estimated cost to reach goal state from n (heuristic)

By applying certain constraints on h(n) A* is both Complete & optimal.

Scanned with CamScanner

P

Greedy best-first Search

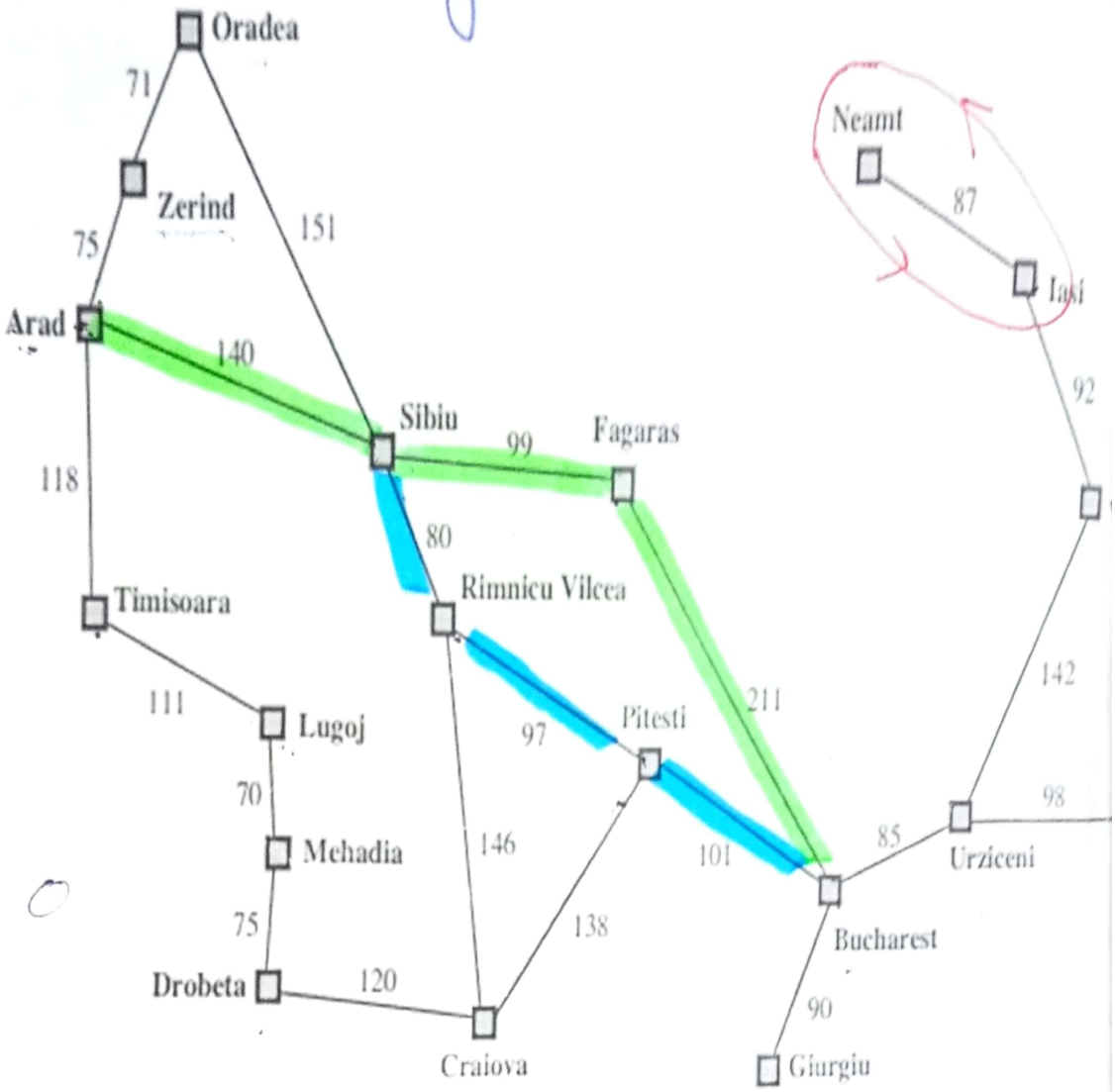


Figure 3.2 A simplified road map of part of Romania.

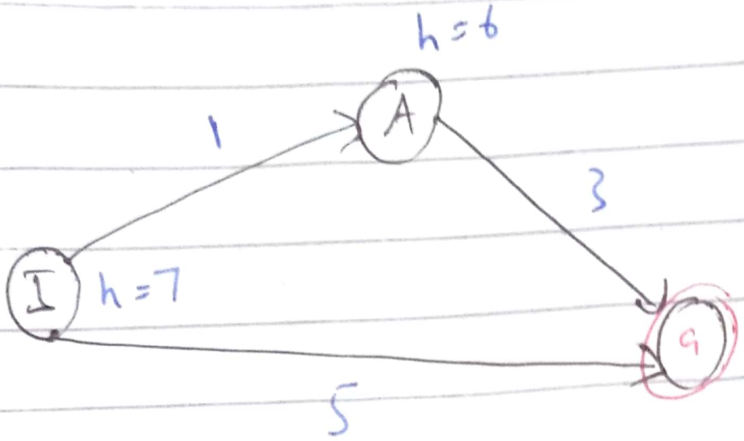
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure 3.22 Values of h_{SLD}—straight-line distances to Bucharest.

Scanned with CamScanner

Example when A* is not optimal

Lets consider the follow graph.



-> Now applying the A* search starting from I

from I -> A , f(n) = 1+6 = 7

I -> G , f(n) = 5+0 = 5

so 5 < 7 so we select 5 ~~so and~~ ~~Frontier~~ ~~but~~ but ~~we~~ as UCS we apply goal test when a node is selected for expansion

I -> A = 7

I -> G = 5

now we select I-G and apply goal test but here we find goal but this solution is not optimal because

Scanned with CamScanner

the optimal solution is

I -> A -> G with cost > 4

but

I - G has cost = 5

so to overcome this our h(n) must be admissible

:: Conditions For Optimality ::

-> For optimality we have two condition

- Admissibility : In tree search
- Consistency : In Graph Seach

:: Admissibility ::

-> h(n) is always on lower bound

-> For optimality in A* search our heuristic must be admissible by definition

An admissible heuristic is the one that never overestimate the true cost to reach the goal

Scanned with CamScanner

mathematically:

$$0 \leq h(n) \leq h^*(n)$$

$h(n)$ = heuristic estimate to reach goal
$h^*(n)$ = actual cost to reach the goal

∴ Proof that it guarantees optimality ∴

→ We can prove that admissible heuristic guarantees optimality

$$f(n) = g(n) + h(n)$$

from def of admissible heuristic

$$h(n) \leq h^*(n)$$

so

$$f(n) < g^*(n)$$

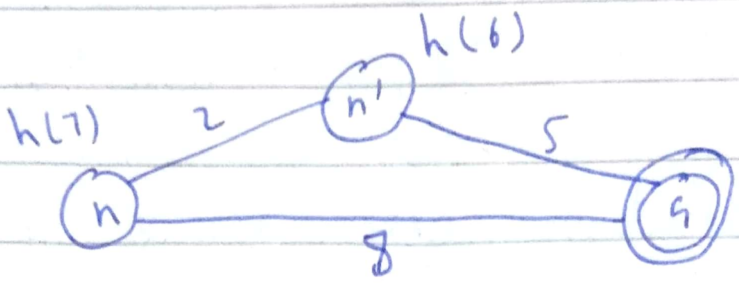
$g^*(n)$ → actl cost to reach goal from initial

hence our solution will be optimal.

Scanned with CamScanner

:: Consistency :-

Consistencealty or monotonicity is a slightly stronger condition of requir for graph search A*.



-> It states that a heuristic is said to be a consistent heuristic if ~~th~~ ~~es~~ ∀ node n and its successors n' achieve by action a, the ~~est~~ heuristic cost to reach the goal from n is always <= to the step cost from n to n' + heuristic cost of reaching goal from n'

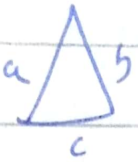
-> Mathematically :

$$h(n) \leq c(n, n', a) + h(n')$$

Scanned with CamScanner

This forms a triangular inequality

i.e ~~sum of~~



$$c <= a+b$$

→ This consistency condition is not required in tree search because in tree search the path to reach a specific node is only one (I say node not state because different nodes may represent same state.) But in graph ∃ multiple paths to reach a node.

→ The consistency condition is also seems logical because if $h(n)$ will be greater than $h(n') + c(n,n',a)$ then it voilates our claim that $h(n)$ is admissible i.e lower bound (never overestimate) the cost to reach the goal.

Scanned with CamScanner

:: Proof that ~~admissibility~~ Consistency guarantees optimality.

-> We will show that when a goal node is selected for expansion then it will be the optimal solution i.e all the next goal nodes are sub optimal.

-> We first prove the following claims.

-> Claim 1: Consistency implies non decreasing f-values along a path

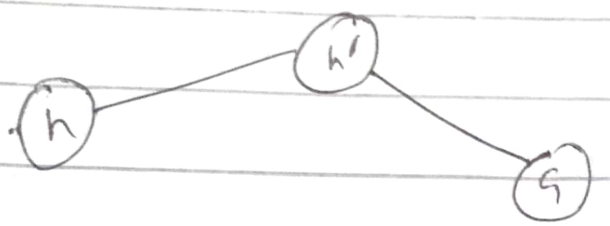
-> Claim 2 : When a node is selected for expansion, an optimal path to that node has been found

-> Claim 3: The first goal node selected for expansion is optimal solution.

Scanned with CamScanner

Claim 1: Consistency implies non dec f-values along a path.

→ Let us ~~c~~ consistency guarantees that the f value of all nodes from node n to goal node will we non-decreasing values.



Let us consider node n whose f value is

$$f(n) = g(n) + h(n) \quad (1)$$

n' is a successor of n by action a and its f-value is

$$f(n') = g(n') + h(n')$$

but

$$g(n') = g(n) + c(n, n', a)$$

Scanned with CamScanner

That is the

path cost to n' = path cost to n + step cost from n to n'

So.

$$f(n') = g(n) + c(n, n', a) + h(n')$$

From definition of a consistent heuristic

$$h(n) \leq c(n, n', a) + h(n')$$

Adding $g(n)$ on b.s

$$g(n) + h(n) \leq g(n) + c(n, n', a) + h(n')$$

①

$$f(n) \leq f(n')$$

Conclusion :-

Hence proved any node on the path from n to goal has a non-decreasing f-value i.e greater or equal to its parent.

Scanned with CamScanner

Claim 2: When a node (n) is selected for expansion, an optimal solution to that node has been found.

→ This claim is true due to claim 1 that every node along a path have non-decreasing f-value. if ∃ a node n' with smaller f-value than n then it must have selected first because f-values are non-decreasing.

Claim 3: When a goal node is expanded it is the optimal solution

→ The first goal that is selected for expansion must be the optimal solution as h = 0

$$f(n) = g(n) + h(n)$$

$$\boxed{f(n) = g(n)}$$

is is due to above two claims

Scanned with CamScanner

all the goals that occur later will have value greater than actual cost to ~~f(n)~~ goal node since f has non decreasing value.

Hence consistent heuristic promises optimal solution.

Scanned with CamScanner